

TRABAJO PRÁCTICO N.º 1

Slay the Spire

El Ascenso a la Aguja

Modelado orientado a objetos de un sistema de rol por turnos en C#

Docente: TUP Albornoz Lucas Ariel

Modalidad	Individual
Lenguaje	C# — Aplicación de consola
Entrega	Discord

1. Contexto

Un personaje debe escalar una torre —la Aguja— enfrentando enemigos, gestionando un inventario de consumibles y acumulando reliquias hasta derrotar al Jefe Final en el último piso.

La cantidad total de pisos se sortea (es decir, se define de manera aleatoria) al iniciar la partida (entero entre 10 y 30 inclusive). El Jefe Final ocupa siempre el último piso generado (`pisoActual == totalPisos`), cualquiera sea ese total.

Debe aplicarse encapsulamiento, composición y, opcionalmente, polimorfismo, con reglas de negocio precisas y sin contradicciones entre el enunciado y el código.

2. Arquitectura de Clases Requerida

2.1 — Item → ItemPocion / ItemReliquia

Un atributo tipo ("Pocion" o "Reliquia") determina el comportamiento de `usar()`. Como alternativa opcional y más avanzada, puede implementarse con herencia: clase abstracta `Item` con subclases `Pocion` y `Reliquia` (ver 6).

ATRIBUTOS

- **nombre** : **string** — nombre descriptivo del ítem.
- **tipo** : **enum/string** — "Pocion" o "Reliquia"; define la semántica de efecto.
- **efecto** : **int** — Pocion → valor instantáneo (vida o ataque temporal). Reliquia → valor permanente sumado a `ataqueBase` o `vidaMaxima` al equiparse.
- **precio** : **int** — costo en oro; 0 si no está a la venta.

MÉTODOS

- **usar(jugador)** — Pocion: aplica el efecto de inmediato y se consume. Reliquia: se invoca una única vez, en `equiparReliquia()`.
- **mostrarDetalle()** — imprime nombre, tipo y descripción breve del efecto.

2.2 — Inventario

Gestiona exclusivamente las pociones del jugador. Las reliquias no se almacenan aquí.

ATRIBUTOS

- **pociones** : **List<ItemPocion>** — capacidad máxima fija (por defecto, 3).
- **capacidadMaxima** : **int** — tamaño máximo de la lista (constante = 3, configurable).

MÉTODOS

- **agregarPocion(pocion)** — agrega si hay espacio; si está lleno, devuelve false y no fuerza el límite (ver 4.1).
- **usarPocion(indice, jugador)** — valida el índice; si es válido, invoca `usar()`, elimina la poción y consume el turno; si es inválido, informa el error sin gastar turno.

- **listarPociones()** — muestra cada espacio numerado; los vacíos se listan como "Vacío", sin reutilizar ese número.

2.3 — Jugador

ATRIBUTOS

- **nombre** : **string**
- **vidaActual**, **vidaMaxima** : **int** — **vidaActual** nunca supera **vidaMaxima** ni baja de 0.
- **ataqueBase** : **int**
- **oro** : **int**
- **pisoActual** : **int**
- **inventario** : **Inventario** — composición: el Jugador "tiene un" Inventario.
- **reliquiasEquipadas** : **List<ItemReliquia>** — lista, no un único objeto: capacidad ilimitada para reliquias.

MÉTODOS

- **atacar(enemigo)** — **daño** = **ataqueBase** + suma de bonus de **reliquiasEquipadas**; se aplica a **enemigo.vida**.
- **equiparReliquia(reliquia)** — agrega la reliquia y aplica su bonus permanente de inmediato. Si aumenta **vidaMaxima**, **vidaActual** se incrementa en la misma proporción.
- **mostrarEstado()** — **vida actual/máxima**, **oro**, **ataque total**, **reliquias equipadas** e **inventario**.

2.4 — Enemigo

ATRIBUTOS

- **nombre** : **string**
- **vida** : **int**
- **puntosAtaque** : **int**
- **oroRecompensa** : **int**
- **posibleDrop** : **Item (opcional)** — ítem obtenible al ser derrotado, sujeto a una probabilidad (ej. 30%).

MÉTODOS

- **atacar(jugador)** — aplica **puntosAtaque** a **jugador.vidaActual**, sin bajar de 0.
- **estaVivo()** — retorna **true** mientras **vida > 0**; controla el fin del combate.

IMPORTANTE Es opcional escalar **puntosAtaque**, **vida** y **oroRecompensa** en función de **pisoActual**, para que el Jefe Final represente siempre un desafío significativo.

3. Flujo del Juego y Eventos por Sala

Al avanzar de piso, el tipo de sala se determina de forma aleatoria, salvo en el último piso, que siempre es el Jefe Final.

Tipo de sala	Comportamiento exigido
Combate	En cada turno: Atacar, Usar Poción o Huir. Usar poción o Huir consume el turno. Al vencer, evaluar posibleDrop.
Tienda / Mercader	Comprar pociones (sujeto a espacio) o reliquias (sin límite de cantidad).
Cofre de Reliquias	Otorga una reliquia que se agrega mediante equiparReliquia().
Fogata (Descanso)	Recupera un % de vidaMaxima (ej. 30%) sin superarla, u omitir el descanso.
Jefe Final (último piso)	Combate obligatorio. No se permite Huir.

4. Ejemplo de Menú de Interacción en Consola

El proyecto debe reproducir un flujo de consola equivalente al siguiente ejemplo.

```

=== SLAY THE SPIRE - CONSOLA ===
Piso: 5/18 | Vida: 42/80 | Oro: 60g
Reliquias equipadas: [Anillo del Valor (+3 Atq)]
Pociones en Inventario (2/3):
  [1. Poción de Vida (+25 HP)]
  [2. Poción de Fuerza (+5 Atq)]

1. Avanzar de sala
2. Abrir Inventario / Usar Poción
3. Ver Estado Completo
4. Salir

Seleccione una opción: 2

--- INVENTARIO DE POCIONES ---
1. Poción de Vida (Curación +25 HP)
2. Poción de Fuerza (Aumenta ataque temporalmente)
3. Vacío
0. Volver al menú

Elija la poción a consumir (0 para volver): 1
[+] Consumiste Poción de Vida. Vida actual: 67/80

```

IMPORTANTE La opción "0. Volver al menú" no debe reutilizar el número de un espacio del inventario. La vida máxima permanece invariable al consumir una Poción de Vida: solo se modifica vidaActual.

5. Reglas de Negocio del Inventario

5.1 — LÍMITE DE CAPACIDAD

- Máximo de espacios para pociones: 3 (por defecto). Las reliquias no están sujetas a este límite.
- Si se recibe o compra una poción con el inventario lleno, el sistema debe ofrecer descartar la poción nueva o cancelar la operación. Nunca se agrega forzando el límite.

5.2 — EFECTO DE LAS RELIQUIAS

- Se almacenan en reliquias Equipadas y son pasivas: su efecto se aplica una única vez, al equiparse, modificando de forma permanente ataqueBase o vidaMaxima.

5.3 — USO DE ÍTEMS EN COMBATE

- Usar una poción consume el turno: tras usarla, el enemigo ataca de inmediato.
- Huir consume el turno y tiene una probabilidad de éxito (ej. 50%); si falla, el enemigo ataca igual.
- Huir no está disponible contra el Jefe Final.

6. Conceptos Pedagógicos Evaluados

- **Composición** — el Jugador "tiene un" Inventario, y el Inventario "tiene" una colección de pociones.
- **Colecciones** — inserción, eliminación, recorrido y validación de límites sobre pociones y reliquias.
- **Encapsulamiento** — propiedades con setters controlados; vidaActual nunca fuera de [0, vidaMaxima] desde afuera de la clase.
- **Polimorfismo (opcional)** — Item como clase abstracta con subclases Poción y Reliquia, cada una con su propio usar().

7. Entrega

- Proyecto en C# (aplicación de consola).
- README con las decisiones de diseño tomadas, en especial si se optó por la variante con herencia (Item / Poción / Reliquia).
- Evidencia de pruebas sobre los casos límite: inventario lleno, vida en los extremos (0 y vidaMaxima), e intento de huir del Jefe Final.